

Weight-Based Privacy-Preserving Asynchronous SplitFed for Multimedia Healthcare Data

VERONIKA STEPHANIE and IBRAHIM KHALIL, RMIT University, Melbourne, Australia
MOHAMMED ATIQUZZAMAN, University of Oklahoma, Norman, OK, USA

Multimedia significantly enhances modern healthcare by facilitating the analysis and sharing of diverse data, including medical images, videos, and sensor data. Integrating AI for multimedia data classification shows promise in improving healthcare services, data analysis, and decision-making. However, ensuring privacy in AI-integrated healthcare systems remains a challenge, especially with data continuously transmitted over networks. Synchronous Federated Learning (FL) is designed to address these privacy concerns by allowing end devices to collaboratively train a machine learning model without sharing data. Nonetheless, FL alone does not fully resolve privacy issues and faces efficiency challenges, particularly with devices of varying computational capabilities. In this article, we introduce an Asynchronous Partial Privacy-preserving Split-Federated Learning (APP-SplitFed) approach for smart healthcare systems. This method reduces computational demands on resource-limited devices and uses a weight-based aggregation method to allow devices of differing computational power to contribute effectively, ensuring optimal model performance and rapid convergence. Additionally, we incorporate a secure aggregation method to prevent adversaries from identifying individual models owned by healthcare institutions.

CCS Concepts: • **Security and privacy** → **Privacy protections**; **Distributed systems security**; • **Computing methodologies** → **Computer vision tasks**; • **Applied computing** → **Health care information systems**;

Additional Key Words and Phrases: Federated Learning, Split Learning, Internet-of-Medical Things, Deep Learning, Secure Aggregation, Secure Multi-Party Computation

ACM Reference format:

Veronika Stephanie, Ibrahim Khalil, and Mohammed Atiquzzaman. 2024. Weight-Based Privacy-Preserving Asynchronous SplitFed for Multimedia Healthcare Data. *ACM Trans. Multimedia Comput. Commun. Appl.* 20, 12, Article 377 (November 2024), 24 pages.
<https://doi.org/10.1145/3695876>

1 Introduction

The importance of multimedia data in healthcare has grown significantly due to technological advancements and the demand for improved healthcare solutions. This data, which includes medical images, videos, and sensor outputs, plays an essential role in the diagnosis, treatment, and ongoing management of patient care. With the integration of **Artificial Intelligence (AI)**, including

This work was supported by the Australian Research Council Discovery Project (DP210102761).

Authors' Contact Information: Veronika Stephanie (corresponding author), RMIT University, Melbourne, Australia; e-mail: veronika.stephanie@student.rmit.edu.au; Ibrahim Khalil, RMIT University, Melbourne, Australia; e-mail: ibrahim.khalil@rmit.edu.au; Mohammed Atiquzzaman, University of Oklahoma, Norman, OK, USA; e-mail: atiq@ou.edu. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1551-6865/2024/11-ART377

<https://doi.org/10.1145/3695876>

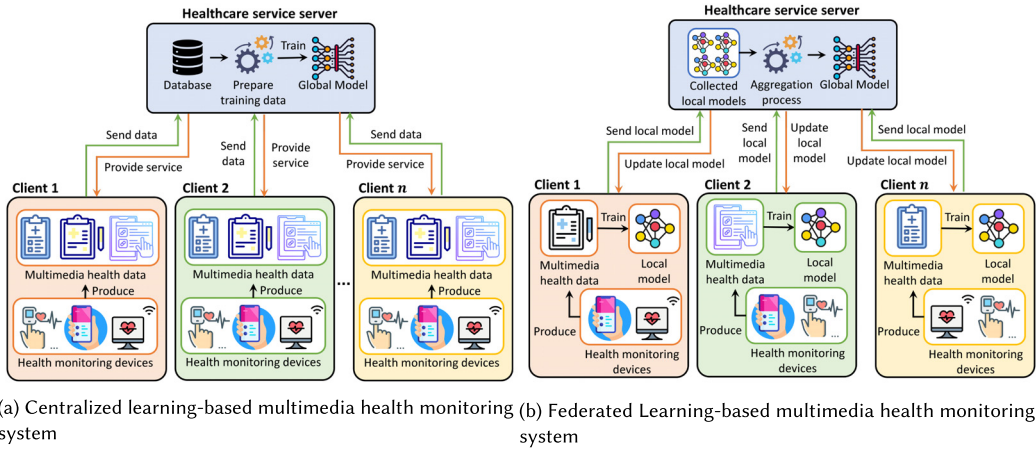


Fig. 1. AI-based multimedia health monitoring architectures.

machine learning and deep learning, healthcare professionals are now equipped with advanced tools to process this data swiftly and with greater precision. This enhancement in data handling improves healthcare delivery, patient outcomes, diagnostic accuracy, and patient care, while also streamlining healthcare processes.

AI-driven classification of multimedia data has shown substantial potential in advancing healthcare services across various domains. These services cover a wide range of applications that positively affect patient care. For instance, automated insulin treatment for individuals with type 1 diabetes, as shown in [27], demonstrates how AI can autonomously adjust insulin dosages based on real-time data, improving glycemic control and enhancing the quality of life for diabetes patients. Additionally, machine learning-based colorectal cancer screening using Colorectal Capsule Endoscopy diagnostic imaging, as highlighted in [7], offers a non-invasive and highly accurate method for early cancer detection, potentially saving lives. Furthermore, the application of machine learning algorithms for **Electronic Health Record (EHR)** analysis, as demonstrated in [4], helps healthcare providers better monitor and manage conditions such as kidney disease, enabling more proactive and personalized interventions.

The conventional centralized learning-based multimedia healthcare monitoring system, depicted in Figure 1(a), comprises a server that gathers multimedia data from various sources, including medical images, EHR, and sensor data. This data is utilized to train an AI model capable of performing inference tasks. Despite the significant potential offered by AI, the employment of centralized learning in multimedia healthcare monitoring systems raises substantial privacy concerns due to the transmission of sensitive data across networks. In such systems, data is transmitted to cloud servers for AI analysis. The reliance on third-party servers, such as cloud servers, introduces privacy risks that could potentially lead to data breaches [37]. Therefore, addressing these privacy challenges is crucial to protect patient privacy and maintain the integrity of the healthcare system.

One promising approach to mitigate these privacy concerns is synchronous **Federated Learning (FL)**, illustrated in Figure 1(b). Synchronous FL represents an effort to balance the benefits of collaborative model training with data privacy. In this framework, each client retains control over their local data, ensuring that sensitive medical information remains within their own secure environments. The process begins with the server distributing model parameters to each client, who then performs local model training using their data. The parameters of these locally trained models are subsequently sent back to the central server for aggregation, resulting in the creation

of a global model. This iterative process continues until a predefined condition is met. While synchronous FL provides a more privacy-conscious alternative to centralized data collection, it is not entirely immune to privacy risks. For example, the study by Nasr et al. [31] demonstrates that successful white-box membership inference attacks can be executed by exploiting parameter updates from locally trained models to reconstruct the data used during training. Thus, even within the FL framework, implementing robust privacy preservation methods is essential to effectively safeguard sensitive healthcare data.

Efficiency is another critical aspect of integrating AI with healthcare systems, particularly given the varying computing capabilities of end devices. This challenge is especially prominent in synchronous FL, where the aggregation process relies on collecting updates from all participating devices to form a unified global model [36, 42]. When a client has less reliable computing and communication resources, their delayed or unsuccessful model updates can disrupt the aggregation process, leading to significant delays in achieving the intended global model convergence. In response to this challenge, researchers have explored asynchronous methods, as discussed in studies such as [10, 14, 26]. These methods aim to address the limitations of synchronous FL by allowing more flexible timing for model updates from clients, thereby reducing the risk of delays caused by resource-constrained devices. However, it is important to note that even these asynchronous approaches may have limitations when dealing with devices that are severely resource-constrained, lacking the computational capacity required to train complex machine learning models effectively.

In recent years, another stream of collaborative learning known as split learning [15] has emerged as a potential solution to address the computing limitations on the client side. Split learning is designed to alleviate the computational burden on individual devices by dividing the model into segments, with each segment processed on different devices. This approach allows more resource-constrained devices to participate effectively in collaborative learning. While split learning holds promise in enhancing the efficiency of AI applications in healthcare, it is not without its own set of challenges. One notable limitation is the scalability of the approach, as coordinating and aggregating data across numerous devices can introduce complexities, potentially impacting the overall effectiveness of the collaborative learning process.

In this article, we introduce an **Asynchronous Partial Privacy-preserving Split-Federated Learning (APP-SplitFed)** approach for smart healthcare systems. **Split-Federated Learning (SFL)**, a core element of APP-SplitFed, alleviates the computational load on healthcare institution devices. We introduce a weight-based aggregation method to ensure effective contributions from devices with varying computing capabilities while considering the performance of client models and delays. Moreover, we integrate a secure aggregation technique to protect against potential adversaries seeking access to individual models owned by healthcare institutions. Our contributions can be summarized as follows:

- We introduce an APP-SplitFed architecture that utilizes a partially distributed training model to reduce communication and end devices computational costs.
- We design a weight-based model aggregation process to accelerate global model convergence.
- We enhance privacy through partial **Secure Multi-party Computation (SMPC)** to mitigate training data reconstruction attacks on known local models.
- The performance of the proposed approach is empirically validated.

The remainder of this article is organized as follows. In Section 2, we present existing work on privacy-preserving collaborative learning methods that address privacy and efficiency issues. In Section 3, we explain our proposed scheme. Section 4 displays the experimental results of our proposed method. Future work is discussed in Section 5. Finally, Section 6 concludes the article.

2 Related Work

Numerous studies have tried to tackle privacy concerns within FL, particularly by integrating FL with privacy preservation techniques such as **Differential Privacy (DP)**, **SMPC**, and **Homomorphic Encryption (HE)**.

The work by Wei et al. [40] advances a privacy-preserving FL framework using DP to obfuscate locally trained model parameters before aggregation, reducing the risk of data leakage. Similarly, Shi et al. [34] employ local DP to perturb clients' models before transmission to the cloud server for aggregation. While FL with DP offers privacy guarantees to a certain extent, there is a tradeoff between privacy and model performance. Better performance leads to lower privacy protection.

To address this tradeoff, Jia et al. [20] combine DP with HE in a proposal for efficient, privacy-preserving, blockchain-based FL in 5G industrial **Internet of Things (IoT)**. Conversely, Zhang et al. [44] present HE-based FL for smart healthcare systems. It involves masking the client's local model before securely transmitting it over the network for aggregation. Unlike DP, HE preserves FL model privacy without significantly affecting model performance. However, it comes with substantial computational overhead.

To enhance the robustness of FL against data reconstruction, multiple studies have integrated SMPC. The work by Kalapaaking et al. [22] introduces an SMPC-based method for securely aggregating FL models. However, this approach requires all clients' involvement in the aggregation process after each training iteration, leading to high communication and computational costs. To address these challenges, Kanagavelu et al. [23] present a committee-based secure aggregation process, aiming to reduce these overheads. Nevertheless, concerns about computational and communication costs remain.

Efficiency in the synchronous FL paradigm has attracted significant research attention. Researchers have explored methods to improve cost-effectiveness, resulting in the development of various asynchronous approaches. Xu et al. [42] categorize these methods into six classifications: node selection-based approaches, weighted aggregation techniques, gradient compression methods, semi-asynchronous strategies, cluster FL methodologies, and model splitting techniques. A summary of the key features, benefits, and limitations of prior works in each category is provided in Table 1.

Within the node selection criterion, Chen et al. [11] propose a heuristic greedy node selection methodology that considers local computation and communication resources for devices to participate in the global learning process. However, this heuristic may not always find the optimal solution and can exclude less reliable devices, leading to potential bias. Meanwhile, Zhou et al. [47] introduce an asynchronous FL protocol for edge computing, optimizing idle edge device utilization while addressing straggler challenges. This asynchronous approach can lead to model inconsistency and requires significant coordination. Hao et al. [16] present a semi-asynchronous FL framework that mitigates straggler impacts in synchronous and asynchronous settings. It uses a data expansion method and a priority function that factors in accuracy and computing power. This semi-asynchronous framework introduces additional complexity and overhead, affecting overall efficiency. Similarly, Wu et al. [41] propose a methodology with client selection and global aggregation to address issues from stragglers, crashes, and model staleness. Although these methods improve model convergence, the node selection process in FL may exclude less reliable nodes, potentially making the learning process less equitable.

In weighted aggregation techniques, Liu et al. [28] and Chen et al. [9] introduce a staleness coefficient in the FL aggregation process to accelerate model convergence, especially with clients of varying computing power and data sizes. However, this may lead to sub-optimal aggregation, with some updates being overemphasized or underrepresented. Similarly, Chen et al. [10] propose

Table 1. Asynchronous FL Approaches Comparisons

Approach	References	Key Features	Benefits	Limitations
Node Selection	[11]	Considers local computation and communication resources for node selection	Improves global learning aggregation process	May exclude less reliable nodes
	[47]	Utilizes idle edge devices, addresses straggler challenges	Optimizes resource usage in edge computing	Requires careful synchronization
	[16]	Incorporates data expansion and priority function for straggler mitigation	Reduces impact of stragglers in synchronous and asynchronous settings	Complexity in managing priorities
	[41]	Introduces strategic client selection to mitigate stragglers and crashes	Enhances model convergence and robustness	Potential equity issues in node selection
Weighted Aggregation	[9, 28]	Introduces staleness coefficient into the FL aggregation process	Fast model convergence in heterogeneous environments	Staleness coefficient might lead to sub-optimal aggregation
	[10]	Proposes an asynchronous partial model update	Reduce communication overhead between server and edge devices	Requires careful management of update cycles
	[19]	Accounts for the age of updates during aggregation	Optimizes aggregation process in asynchronous settings by accounting for the age of updates	Bias may be introduced against less capable devices, leading to a sub-optimal model
	[13]	Utilizes update quality and reverse auction for node participation	Encourages high-quality, low-cost node participation	Requires sophisticated incentive mechanisms
Gradient Compression	[25]	Includes gradient clipping strategy	Reduces communication overhead while preserving model performance	Might impact convergence rates
	[26]	Utilizes double-end sparse compression strategy	Reduces communication overhead while preserving model performance	Performance depends on data distribution and network characteristics
Semi-Asynchronous	[10]	Proposes an asynchronous partial model update	Reduces communication overhead between server and edge devices	Potential synchronization challenges and slower convergence rate
	[32]	Implements aggregation synchronization with different node time frames	Reduces idle time, improves overall efficiency	May introduce bias training
Clustering	[45]	Groups nodes based on gradient direction and latency	Enhances the quality of the global model	Potentially leading to unfair training
	[29]	Cluster clients based on model parameters similarities	Improves convergence and learning process	Introduces additional overhead and require increased storage capability
Model Splitting	[10, 39]	Segmented learning based on node characteristics	Beneficial for resource constrained and heterogeneous environments	Complexity in managing segmented groups and different aggregation frequencies

an asynchronous partial model update mechanism to reduce communication overhead between the central server and edge devices. Nevertheless, the proposed asynchronous updates can cause inconsistencies in the global model, affecting its accuracy and reliability. To address update asynchrony challenges, Hu et al. [19] offer an age-aware weighting design for model aggregation, enhancing

performance in asynchronous FL by considering model age. However, this can introduce bias, with less capable devices contributing less to the final model. While Deng et al. [13] do not explicitly use weighted aggregation for asynchronous updates, it aims to enhance model convergence by estimating update quality and incentivizing high-quality, low-cost node participation through a quality-aware incentive mechanism and an auto-weighted aggregation algorithm. This approach requires careful consideration in building the incentive mechanism.

Recent research, including Koloskova et al. [25], highlights a growing interest in gradient compression techniques. This includes bounding the norm of applied gradients, which involves gradient clipping. For distributed optimization with heterogeneous objectives, the article introduces a convergence rate that considers the average delay within each worker, improving upon previous results that relied more on maximum delay. However, this method may suffer from information loss due to the clipping process, which eventually may impact the global model convergence rate. Additionally, Lee and Lee [26] present semi-asynchronous models to enhance asynchronous FL, featuring varied gradient update mechanisms and a double-end sparse compression strategy aimed at reducing communication overhead while maintaining model performance. However, the effectiveness of these strategies may depend on the specific characteristics of the data distribution and network conditions, potentially limiting their applicability in certain contexts.

The semi-asynchronous model update presents an alternative strategy. In Chen et al. [10], the authors propose asynchronous model updates and temporally weighted aggregation of local models, dividing the learning model into shallow and deep components. The less frequent update of the deep model aims to reduce client-server communication, particularly beneficial for devices with limited bandwidth. However, asynchronous learning may encounter synchronization challenges and slower convergence rates compared to synchronous methods. Moreover, its effectiveness varies depending on network characteristics and the distribution of computing resources among clients. In contrast, Nguyen et al. [32] introduce a buffered asynchronous aggregation method for FL. This method combines the efficiency of synchronous and asynchronous approaches while ensuring compatibility with privacy-preserving technologies. It allows users to send their local model to the server buffer for aggregation, potentially resulting in faster updates from more capable devices, thereby introducing training bias.

Clustering FL as explored in [45] introduces a Clustered Semi-Asynchronous Federated Learning framework that groups nodes based on gradient direction and latency to improve training performance. This approach enhances the quality of the global model by limiting the model staleness. However, it may exclude models that are too stale, potentially leading to unfair training. Long et al. [29] introduce a multi-center FL approach that clusters clients to a cluster center based on the similarity of their learning model parameters. This method improves the learning process by leveraging the similarities among client models and the cluster centres. However, it incurs additional overhead and requires increased storage capability.

Wang et al. [39] and Chen et al. [10] introduce model splitting techniques that involve cascade training with bottom and top subnetworks and differential updating of parameters in shallow versus deep layers. These approaches highlight the move towards more specialized and resource-aware machine learning paradigms. However, they may require more complex training processes and careful management of computational resources.

Beyond asynchronous FL paradigms, several studies have aimed to balance resource allocation among heterogeneous devices by reallocating resources to those with more pronounced needs. Notably, the research by Lu et al. [30] introduces an efficient asynchronous FL framework seamlessly integrated with **Digital Twin (DT)** technology in the context of the Industrial IoT. This approach employs DT technology to enable real-time monitoring of IoT device states, thereby facilitating strategic resource allocation to specific devices. This resource allocation strategy is designed to

effectively reduce communication overhead. However, a persistent challenge in this framework is the enduring presence of aggregation staleness. Efforts aimed at expediting processes in less reliable devices inadvertently introduce delays in more robust devices, underscoring the intricate nature of achieving optimal resource balance within heterogeneous device environments.

Furthermore, existing methods often neglect to account for the computational capabilities of resource-constrained devices. When dealing with large learning models, conventional approaches necessitate each client to complete substantial computations, potentially causing significant staleness in the system or even rendering model training infeasible due to limited storage and computing capacity. To address this challenge, Thapa et al. [38] introduce the concept of SFL, which enables clients to distribute the computational load with the central server. Notably, this article introduces asynchronous model updates within the SFL framework and enhances privacy by implementing secure partial aggregation.

3 Proposed Framework

In this section, we begin by providing an overview of our proposed framework. Subsequently, we elaborate on the proposed work through three subsections: APP-SplitFed, partial weighted model aggregation, and secure partial client model aggregation.

3.1 Architecture Overview

The APP-SplitFed framework is illustrated in Figure 2. In this setup, we consider a scenario where N hospitals, denoted as $\mathcal{H}$, participate in the APP-SplitFed system. Each hospital operates a set of multimedia health monitoring devices $\mathcal{C}$. These devices generate healthcare multimedia data, which may include patient healthcare images, sensor data, or healthcare records. The devices are connected to privately owned edge servers $\mathcal{E}$. Each hospital may possess one or more $\mathcal{E}$, with varying computing capabilities and specifications.

During the training process, K clients are randomly selected from the pool of edge servers $\mathcal{E}$ across all participating hospitals $\mathcal{H}$. Each selected edge server utilizes training data collected from healthcare multimedia sources $\mathcal{C}$ to train the initial layers of a learning model. After processing the multimedia data, the outputs are transmitted to the main server to be fed into the remaining layers of the learning model sequentially. Subsequently, both the remaining layers and the initial layers update their parameters based on the model performance evaluated during the training process.

When a set of initial layers is updated and received by the cloud server within the same buffer time, the corresponding edge servers $\mathcal{E}$ join together to perform a secure aggregation process. The resulting aggregated initial layers are then sent back to the hospital for the next training process.

3.2 APP-SplitFed

In APP-SplitFed, a learning model is divided into two categories, as shown in Figure 3: the initial layers denoted as w_i , and the remaining layers denoted as w_r . Unlike FL, where all computational tasks of training a local model are the responsibility of each client, APP-SplitFed ensures effective distribution of computational tasks for resource-constrained devices by sharing them between hospitals' edge servers and the cloud server. The computation of the initial layers is performed on the hospital's edge server side, while the remaining layers reside on the cloud server side. The sizes of the initial layers ($|w_i| = S_i$) and remaining layers ($|w_r| = S_r$) are typically such that $S_i \ll S_r$, which reduces the computational load on hospitals' resource-constrained devices.

The APP-SplitFed workflow is defined using two algorithms: Algorithm 1 and Algorithm 2. Algorithm 1 outlines the APP-SplitFed process on the client side, where the clients refer to the hospitals' edge servers $\mathcal{E}$ selected to participate in the current round. Each client possesses its

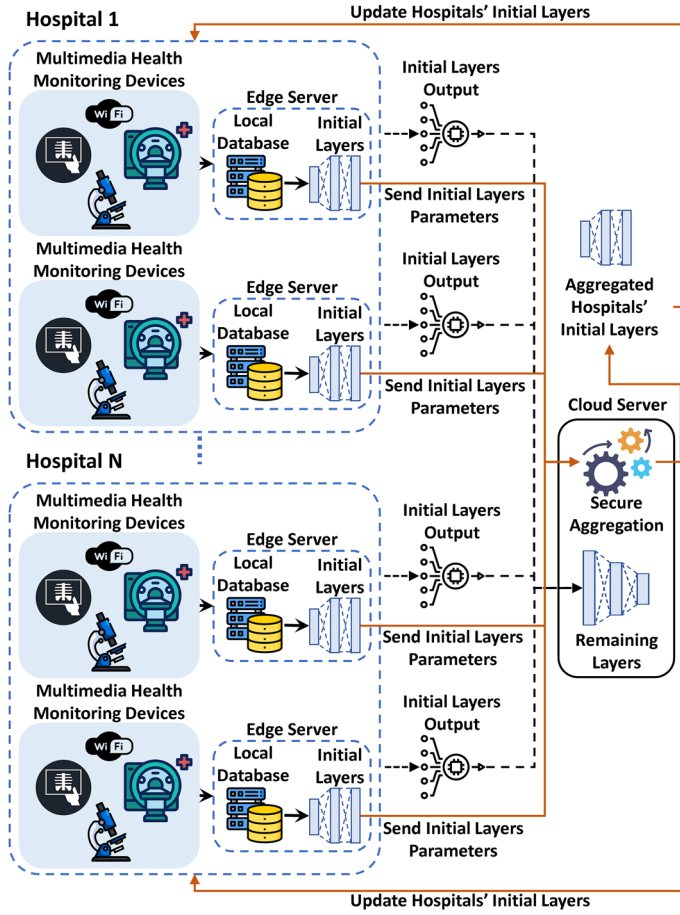


Fig. 2. Architecture overview of APP-SplitFed in smart healthcare system.

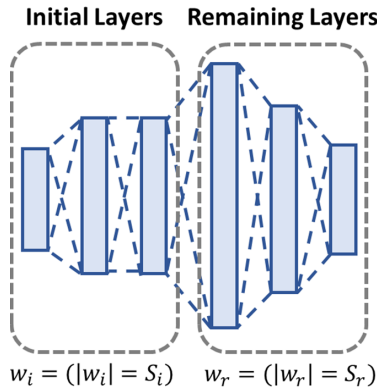


Fig. 3. Model split in APP-SplitFed.

Algorithm 1: APP-SplitFed Client**Input:**Global initial layers, w_i Current server steps, t **1 Initialization:**2 Client's initial layers, $w_{i,e}^p \leftarrow w_i$ 3 Local steps $t_p \leftarrow t$ 4 Local epochs, ep 5 Training status, $finishedTraining \leftarrow False$ **6 begin****7 for** $e \in ep$ **do**8 Forward propagate with data X_p to layers $w_{i,e}^p$ to get intermediate output $\hat{O}_p$ 9 Send $\hat{O}_p$ and data labels Y_p to server10 Receive server calculated gradient $g_{r,e}^s$ 11 Backpropagation using $g_{r,e}^s$ to get client gradient $g_{i,e}^p$ 12 Update initial layers, $w_{i,e+1}^p \leftarrow w_{i,e}^p - \eta g_{i,e}^p$ 13 Set $finishedTraining \leftarrow True$ **14 endfor**

distinct healthcare multimedia data X_p and associated labels Y_p , obtained from its respective set of multimedia health monitoring devices.

During the training process, each client retrieves the global initial layers w_i and the current step t . The step t indicates how many times the w_i has been updated through aggregation processes. Initially, each client sets its local initial layer parameters $w_{i,e}^p$ equal to the global initial layers parameters w_i , and its local step t_p equal to the current step t . The variable t_p remains constant throughout the local training process until aggregation is performed by the clients. Initialization also defines the total number of epochs ep and the client's training status $finishedTraining$.

To contribute to the training process in APP-SplitFed, each client feeds X_p into $w_{i,e}^p$ to produce an intermediate output $\hat{O}_p$. This process runs in parallel across all selected clients. Subsequently, each client sends its $\hat{O}_p$ sequentially to the cloud server for processing. Upon completion of processing each client's output, the cloud server sends back a set of gradients $g_{r,e}^s$ to the respective clients. Only upon receiving $g_{r,e}^s$ does the client proceed with backpropagation using $g_{r,e}^s$ to produce the client gradient $g_{i,e}^p$. This gradient is then used to update the local initial layers: blue $w_{i,e+1}^p \leftarrow w_{i,e}^p - \eta g_{i,e}^p$, where η denotes the learning rate. Finally, the $finishedTraining$ flag is set to *True* to notify the cloud server that the client has completed training.

The APP-SplitFed workflow on the cloud server side is shown in Algorithm 2. The process starts with the cloud server initializing the global initial layer parameters w_i , the remaining layer parameters w_r , and the number of rounds to determine the total aggregation process in the training process. A time buffer T_b must also be specified during the initialization process. This buffer defines when the cloud server needs to inform the clients who have finished their training to aggregate their local initial layers. After the initialization, the cloud server needs to identify the set of clients $\mathcal{E}$ that are registered to participate in the smart healthcare services. Then, K clients are randomly chosen from the list of $\mathcal{E}$. The chosen clients P must finish the previous training process before they can be selected for the next process. The cloud server then requests each client $p \in P$ to run a local training process on their side (see Algorithm 1) in parallel. During this process (step 15–22), p is not required to finish the operation within the current time buffer. Hence, when the time buffer

Algorithm 2: APP-SplitFed Server

```

1 Initialization:
2 Global Initial layers,  $w_i$ 
3 Remaining layers,  $w_r$ 
4 Rounds,  $R$ 
5 Time buffer,  $T_b$ 
6 Current time,  $T$ 
7 Current steps,  $t = 0$ 
8 Number of clients,  $K$ 
9 Clients,  $P = []$ 
10 Hospitals edge servers,  $\mathcal{E}$ 
11 begin
12 for each  $r \leq R$  do
13   if  $T \bmod T_b = 0$  then
14      $P \leftarrow \text{randomClient}(\mathcal{E}, k - |P|)$ , where  $\mathcal{E} \notin P$ 
15     for each  $p \leq P$  in parallel do
16        $p.\text{runProcess}(w_i, t)$ 
17       Receive  $(\hat{O}_p, Y_p)$  from  $p$ 
18       Forward propagation with  $\hat{O}_p$  on  $w_r$  to compute output prediction  $\hat{Y}$ 
19       Calculate loss using  $l(Y_p, \hat{Y})$ 
20       Backpropagation calculate  $\nabla_{\Theta_{w_r}} l(Y_p, \hat{Y})$  to get  $g_{r,e}^s$ 
21       Update remaining layers:  $w_{r+1} \leftarrow w_r - \eta \cdot \frac{1}{N} \sum_{i=1}^N \nabla_{w_r} l(y_{p_i}, \hat{y}_i)$ , where  $N = |Y_p|$ ,  $\eta$  is the
         learning rate,  $y_{p_i} \in Y_p$ , and  $\hat{y}_i \in \hat{Y}$ 
22     endForEach
23     for each  $p \leq P$  in parallel do
24        $\text{aggregateList} = []$ 
25       if  $p.\text{finishedTraining}$  then
26         Remove  $p$  from  $P$ 
27          $\text{aggregateList.append}(p)$ 
28       end
29     endForEach
30     Update global initial layers:  $w_i = \text{aggregateList.aggregate}()$ 
31     Update current steps,  $t = t + 1$ 
32   end
33 endForEach
34 end

```

is reached, the operation can continue. When an intermediate output $\hat{O}_p$ and a set of data labels Y_p are received from a client p , a set of predictions $\hat{Y}$ is obtained by calculating $\hat{O}_p$ on w_r . Next, a loss function $l()$ calculates the model loss using Y_p and $\hat{Y}$. This is followed by a backpropagation process to get the remaining layers' gradients $g_{r,e}^s$. Finally, w_r is updated using Equation (1).

$$w_{r+1} \leftarrow w_r - \eta \cdot \frac{1}{N} \sum_{i=1}^N \nabla_{w_r} l(y_{p_i}, \hat{y}_i). \quad (1)$$

Here, N is the total number of inputs used during the local training, η is the learning rate, $y_{p_i} \in Y_p$, and $\hat{y}_i \in \hat{Y}$.

Algorithm 3: FedAVG Server

```

1 Server Executes:
2 Initialization:
3 Global model,  $w_0$ 
4 Rounds,  $R$ 
5 Number of clients,  $K$ 
6 Clients,  $P = []$ 
7 Hospitals edge servers,  $\mathcal{E}$ 
8 for  $r \in R$  do
9    $P \leftarrow \text{randomClient}(k, \mathcal{E})$ 
10  for each  $p \in P$  do
11     $w_{r+1}^p \leftarrow p.\text{updateClient}(p, w_r)$ 
12  endForEach
13   $w_{r+1} \leftarrow \sum_{p=1}^P \frac{np}{N} w_{r+1}^p$ 
14 endfor

```

When a time buffer is reached by continuously checking if $T \bmod T_b = 0$, the cloud server will contact each p to check if their local training has finished by inspecting the *finishedTraining* flag. If the flag is true, then p needs to proceed to the aggregation process, as explained in Section 3.3. Finally, the current step, which determines the total number of aggregation processes, is incremented.

3.3 Partial Weighted Model Aggregation

To define our proposed partial weighted model update, we first define FedAVG [33] in Algorithm 3 and Algorithm 4, which is a FL aggregation technique used as a fundamental aspect in our work.

The process of FedAVG is similar to APP-SplitFed. However, there is one major difference that can be seen. No further training is done on the cloud server side since the global model is not split for a shared training process. Hence, in the cloud server side (see Algorithm 3), there are only three major processes, namely, random client selection (step 9), client updates (step 11), and the aggregation process (step 13). The aggregation process is explained in Equation (2).

$$w_{r+1} \leftarrow \sum_{p=1}^P \frac{np}{N} w_{r+1}^p. \quad (2)$$

Here, w_{r+1} is the global model update, P is a list of clients, n_p is the number of inputs used by each client $p \in P$ for its local training, N is the total number of data points from all clients P participating in the training, and w_{r+1}^p is the local model update for client p .

On the other hand, as shown in Algorithm 4, the client side is responsible for the entire model training process. The synchronous FL process introduces several challenges, as illustrated in Figure 4(a). Consider a scenario with five clients participating, each possessing varying computational and communication capabilities, operating under a time constraint T . Here, T_1 marks the start of the FL process, and T_6 represents the time when the FL concludes its process in a communication round with an aggregation process. As depicted, clients with limited resources (e.g., client 4) contribute to delays in the aggregation process. In synchronous FL, the cloud server must receive models from all clients before aggregation can commence.

Additionally, the diagram illustrates that each client perform the entire training process. Consequently, clients with lower computation and communication capabilities experience more significant delays compared to those with more resources. This disparity also implies that clients do not fully

Algorithm 4: FedAVG Client

```

1 Client Executes:
2 Initialization:
3 Client's data,  $X_p$ 
4 Client's data labels,  $Y_p$ 
5 Client's local epoch,  $ep$ 
6 Client's local model,  $w_p$ 
7 updateClient( $p, w_r$ ):
8 for each  $e \in ep$  do
9    $\hat{Y} \leftarrow w_p(X_p)$ 
10   $w_{p+1} \leftarrow w_p - \eta \cdot \frac{1}{N} \sum_{i=1}^N \nabla_{w_p} l(y_{p_i}, \hat{y}_i)$ , where  $y_{p_i} \in Y_p$  and  $\hat{y}_i \in \hat{Y}$ 
11 endForEach

```

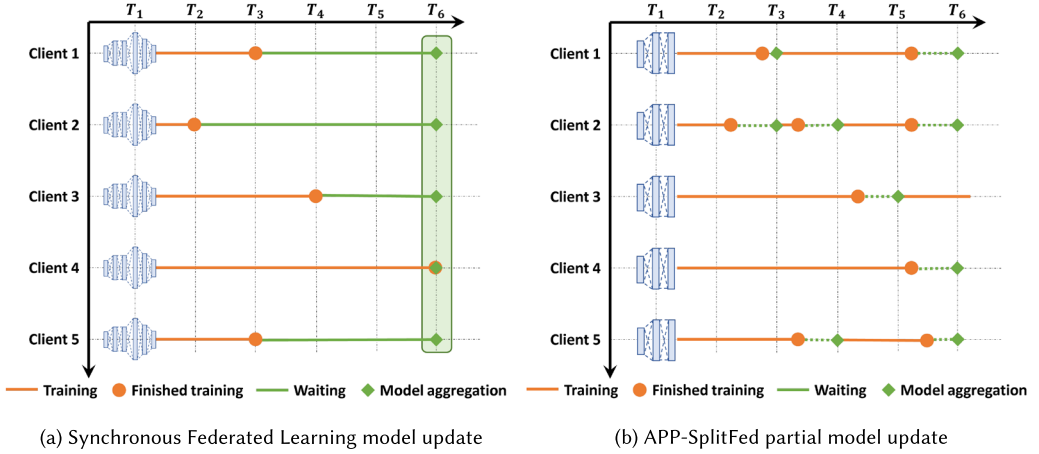


Fig. 4. Synchronous and asynchronous model updates.

leverage the computational power available on the cloud server, which typically exceeds that of client devices.

For this purpose, we introduce an asynchronous method in APP-SplitFed. As shown in Figure 4(b), we allow clients who have completed their training process within a specified time buffer (e.g., from T_2 to T_3) to aggregate their models early. Consequently, the global model can be updated promptly with the most recent training data, and the next client can be selected immediately for subsequent training. It is also noteworthy that the integration of SFL and FL enables clients to train the model partially. In this scenario, the initial layers of the model w_i , which are relatively small in size, are trained on the client's side. This approach alleviates computational costs on the client side while leveraging the computational resources of the cloud server.

In FL and SFL, all trained local models possess valuable information regardless of when they are aggregated. However, under the assumption of Independent and Identically Distributed data and an equal number of inputs used for training each local model, a model trained with the most recent global model parameters holds more valuable information than one trained with less recent parameters. Therefore, authors in [10, 36] proposed a time-aware aggregation method that considers the time difference between the global model updates used during training and the time of the

aggregation process (see Equations (3) and (4)).

$$D_p = \frac{np}{N} * \left(\frac{e}{2}\right)^{-(t-tp)} * (w_i - w_{r+1}^p), \quad (3)$$

$$w_{i+1} = w_i - \sum_{p=1}^P D_p. \quad (4)$$

Here, D_p represents the weighted parameter updates considering both the number of input data and time. w_{i+1} denotes the updated global initial layers, while w_i stands for the global initial layers. P denotes the set of clients, np indicates the number of inputs used by client p , N represents the total number of data points used in training by all clients in P , e denotes the natural logarithm, tp records the steps when a client first receives model parameters from the cloud server, t denotes the current step when client p joins the aggregation, and w_{r+1}^p signifies the update of the local initial layers.

The natural logarithm in this equation augments the existing FedAVG aggregation method in Equation (2) to incorporate the temporal aspect in asynchronous model updates. This adjustment ensures that models with more recent information contribute more significantly to the final models. However, prior works such as those by [10, 36] assume that models with more recent updates inherently outperform their predecessors, overlooking the possibility that certain clients may possess superior data representations and thus yield better performance. Essentially, these studies neglect the importance of evaluating model performance. To address this gap, we propose a delay-based aggregation method that considers client model performance, as defined in Equation (5).

$$D_p = \frac{np}{N} * \left(\frac{e}{2}\right)^{-\frac{(\mathcal{A}-a_p)}{\mathcal{A}}(t-tp)} * (w_i - w_{r+1}^p). \quad (5)$$

Here, $\mathcal{A}$ refers to the maximum accuracy of all clients, and a_p is the accuracy of client p . The additional matrix enables client model performance to offset the contribution, initially determined solely by the client's delay.

3.4 Secure Partial Client Model Aggregation

Synchronous FL local models are considered to contain information that can lead to training data reconstruction [21]. The same threats apply to the APP-SplitFed learning model when clients' initial layers are sent to the centralized server for aggregation. Information leakage from the gradient of a learning model, particularly the gradient of the first layer, is a significant factor in successful training data reconstruction attacks. Sotthiwat et al. [35] have demonstrated that performing a SMPC-based aggregation on the first layer's gradients can mitigate gradient-based data reconstruction techniques such as **Deep Leakage from Gradient (DLG)** [48] and Improved DLG [46]. Inspired by Sotthiwat et al. [35], we propose a secure partial aggregation using SMPC for APP-SplitFed.

We used the additive secret sharing protocol introduced in [12] for our secure aggregation method. Figure 5 illustrates the process of multiple parties participating in additive secret sharing computation. Suppose there are a server S and edge servers $\mathcal{E}_n$ involved in APP-SplitFed. Each entity has its value that needs to be summed. In our case, the value is the parameters of the initial layers obtained after each client computes D_p from Equation (5). Since the secret sharing technique accepts integer numbers, the value needs to be converted from floating points to integers. For this, we used fixed precision encoding, which allows us to store decimal values as approximate values using n -bit integers. Before creating a share, each client receives a large prime number Q from the

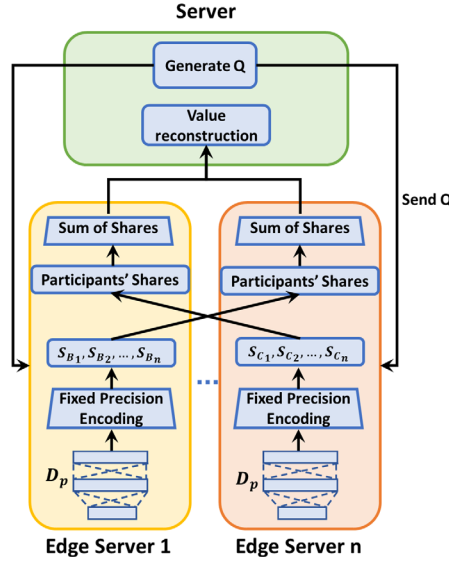


Fig. 5. Secure aggregation method for global initial layers.

cloud server. Then, each client generates n shares $S_1, S_2, \dots, S_n$ of their own D_p . These shares must satisfy:

$$S = \left(\sum_{i=1}^n S_i \right) \bmod Q. \quad (6)$$

Here, S is the real value to be shared, n is the number of shares, and S_i represents the shares to be distributed to n clients. These shares are then distributed to all clients participating in the additive secret sharing protocol. After the clients receive the shares, they sum them up and send the results to the cloud server for value reconstruction. However, since the first layer contains important information that can be used for training data reconstruction, the first layer of $w_{i,e+1}^p$ is not sent to the cloud server. Instead, it remains on the client's side. The cloud server then continues its process to update the global initial layers (see Equation (4)).

4 Results and Discussion

This section provides information on the testing environment and datasets used for the experiments. Performance evaluation and analysis of our proposed scheme are also presented.

4.1 Testing Environment

We used a Windows 10 personal computer with 16 GB of RAM, an AMD Ryzen 9 CPU, and an NVIDIA GeForce RTX 3060 GPU. The programs were written using Jupyter Notebook with Python version 3.8.

4.2 Datasets and Model

In this experiment, we utilize well-established multimedia health datasets often used as benchmarks. We focus on two main categories of multimedia healthcare data: medical images, including BloodM-NIST and PathMNIST, and medical time series data, such as MHealth and **Electroencephalogram (EEG)** Brainwave data. The four datasets used in the experiments are described as follows:

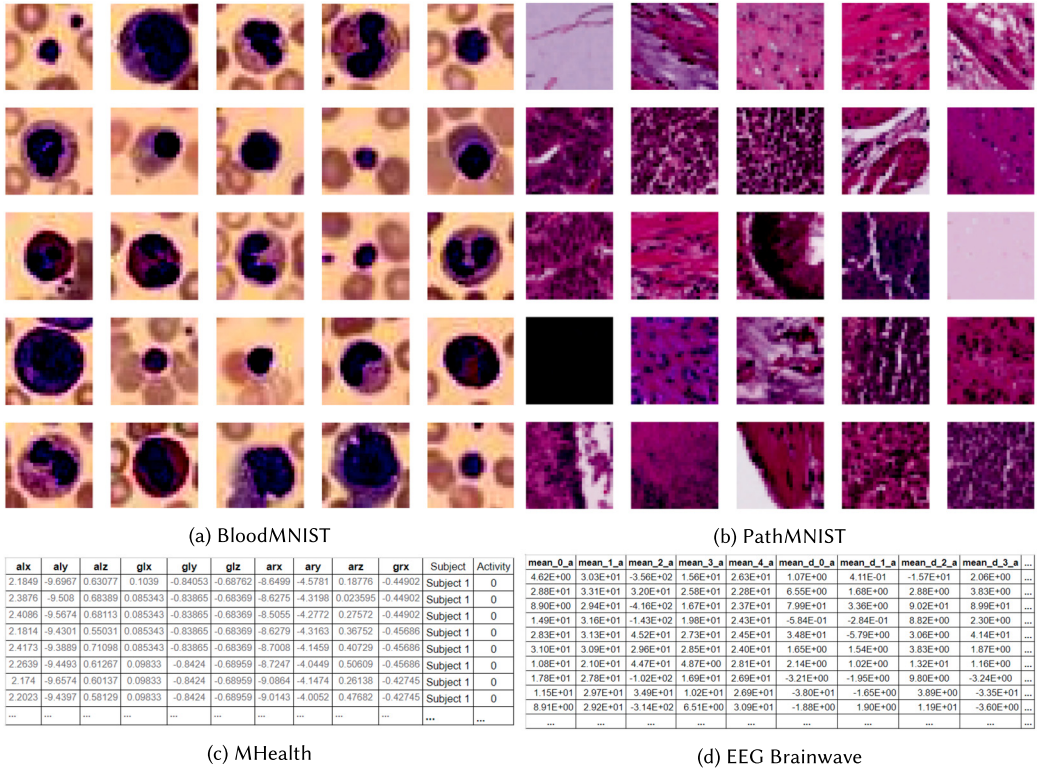
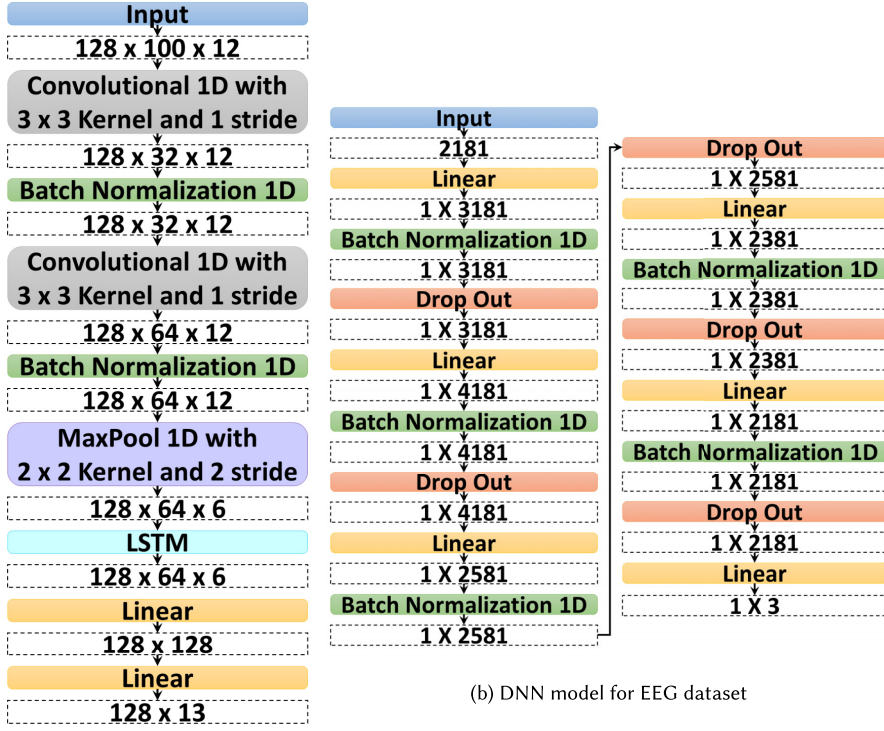


Fig. 6. Multimedia health datasets overview.

- BloodMNIST* (see Figure 6(a)). This dataset is published by [1, 43]. It consists of 17,092 images with 3 color channels and a resolution of 28×28 pixels, depicting blood cells from individuals with hematologic or oncologic diseases, as well as uninfected individuals. The images are categorized into eight classes for multi-class classification tasks and divided into 11,959 training, 1,712 validation, and 3,421 testing sets.
- PathMNIST* (see Figure 6(b)). PathMNIST is also a dataset published by [24, 43]. This dataset is designed to benchmark techniques for disease detection in colon pathology using standardized human colon images. Each image is pre-processed to 28×28 pixels with 3 color channels and classified into nine tissue types. The dataset provides 89,996 training, 10,004 validation, and 7,180 testing samples for training and testing machine learning models.
- MHEALTH* (see Figure 6(c)). MHEALTH (Mobile Health) [2, 3] is a time-series dataset used for human behavior analysis based on body motion and vital signs (such as acceleration, body part angle, and rotation). The data were collected from 10 volunteers performing 12 activities or remaining idle.
- EEG Brainwave* (see Figure 6(d)). The EEG Brainwave dataset [5, 6] consists of EEG data aimed at detecting human emotions, including negative, positive, and neutral states. The data were collected from two individuals over six minutes using EEG headbands. Participants were exposed to emotionally engaging movies to capture brainwaves of particular interest.

We utilized three learning models to test the performance of our proposed work. ResNet18 [17] was used for the BloodMNIST and PathMNIST datasets. The MHEALTH dataset was tested using a



(a) LSTM-CNN model for MHEALTH dataset

(b) DNN model for EEG dataset

Fig. 7. Machine learning model architectures.

Long Short-Term Memory [18] and a one-dimensional Convolutional Neural Network, depicted in Figure 7(a). For the EEG dataset, a simple Deep Neural Network was employed, with the learning model defined in Figure 7(b).

We define a consistent partition to ease the experiments. Specifically, one-third of the layers are designated as initial layers, while the remaining layers reside on the cloud server.

4.3 Results and Performance Evaluation

In this section, we analyse the effect of delay dispersion among clients on the performance of the global model and compare it with prior work in [36]. We conducted experiments using two clients. To examine whether delay dispersion causes performance degradation in the global model, we assigned an integer value a to one client. For the other client, we assigned a value of $a + d$, where d represents the number of additional rounds before the client can participate in the aggregation process. For instance, a client with delay a participates in each round of aggregation, while a client with delay $a + d$ (where $d = 3$) aggregates its model every three rounds. In Equation (5), this delay is represented as the difference between t_p and the current t .

Figure 8 showcases the performance of the prior work described by Stephanie et al. [36] across various multimedia healthcare datasets. In contrast, Figure 9 displays the results of our proposed method on multiple multimedia healthcare datasets. Both experiments were conducted under varying values of d .

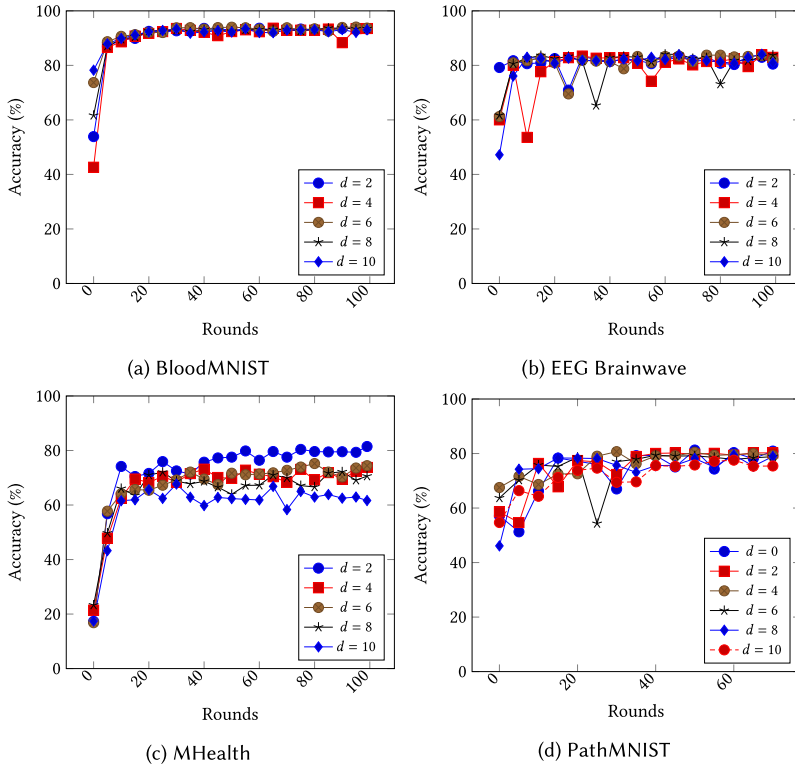


Fig. 8. Impact of delay dispersion (2 to 10 units) on model performance in prior work [36] across various multimedia health datasets.

In the analysis of the prior work, an inverse relationship between higher values of d and global model accuracy was observed, although this trend was not consistently significant across all datasets. This pattern is most evident in Figure 8(c) and (d), where longer client delays reduce the information contribution from clients with limited resources. Conversely, in our proposed method, shown in Figure 9, client delay dispersion does not significantly impact overall model performance.

This improvement is due to an additional control variable, outlined in Equation (5), which accounts for each client's performance. Thus, regardless of delays, clients with comparable model performance can contribute equally to the collaborative model. This control mechanism ensures sustained client involvement and equal contribution.

We analysed the performance of our proposed method against state-of-the-art techniques using various **Neural Network (NN)** models on four multimedia health datasets in smart healthcare systems. The experiment involved 30 clients, each assigned an integer value d to determine the timing of aggregation. To ensure fairness, we enforced uniform delay settings for all clients in the asynchronous schemes.

Figure 10 compares the performance of our proposed method to other existing methods, namely, SFL [38] in synchronous and asynchronous setups, and layer-wise asynchronous FL [10]. As can be seen, synchronous SFL produces the highest accuracies compared to other methods. This is because synchronous SFL receives updates from all the chosen clients regardless of their delay. However, each communication round in synchronous SFL may require more time than the asynchronous setup when devices with heterogeneous computing and communication capabilities are involved.

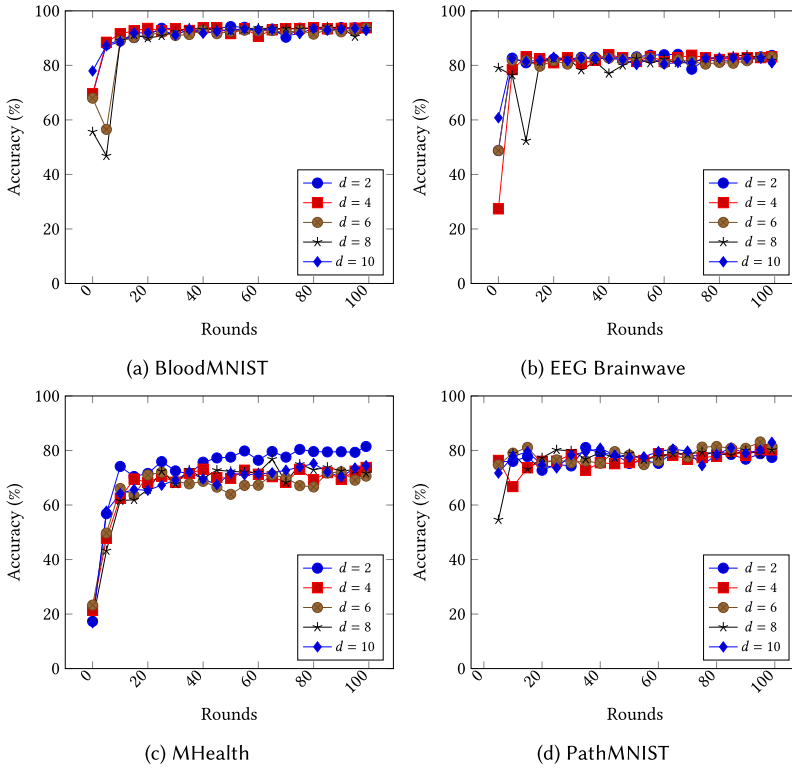


Fig. 9. Impact of delay dispersion (2 to 10 units) on model performance in proposed work across various multimedia health datasets.

Our method also shows competitive performance compared to synchronous SFL while introducing asynchronicity to the system, allowing efficient resource usage. Clients with more capability can continue their process without being delayed. Additionally, the weighted aggregation method enables fast model convergence and higher model performance, which can be clearly seen in Figure 10(c) and (d).

On the other hand, when delays in the clients are introduced in SFL without a weighted update mechanism, high fluctuation can be seen in the early rounds before the model converges. The performance of the layer-wise asynchronous method falls slightly below all other methods. This is because the method requires early layers to be aggregated less frequently. However, since the early layers of the model contain important information, a delay in their aggregation may cause late model convergence.

We then conducted an evaluation of the privacy preservation method's performance under a range of conditions, focusing on the number of securely aggregated layers and the number of clients involved. In particular, we analyzed the time required for share generation and model aggregation processes. For this experiment, we incremented the number of layers by one-third of the total number of layers for each machine learning model. The experiments were conducted using 10, 20, 30, 40, and 50 clients.

As illustrated in Figure 11, a consistent trend is observed, indicating that the number of layers aggregated on the client side significantly influences the total time required to generate the shares. Specifically, an increase in the number of layers correlates with a longer time to generate the shares.

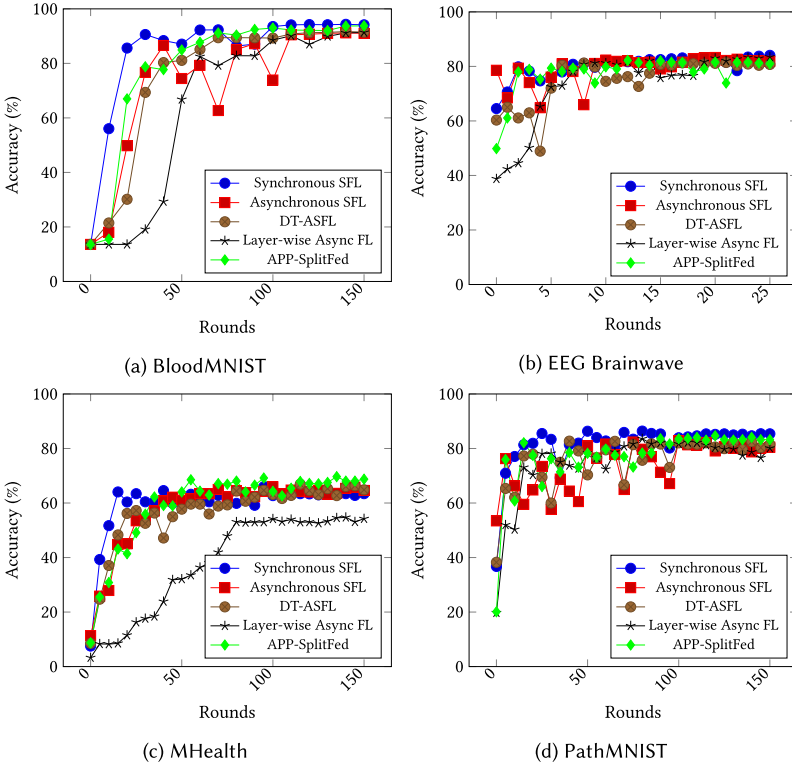
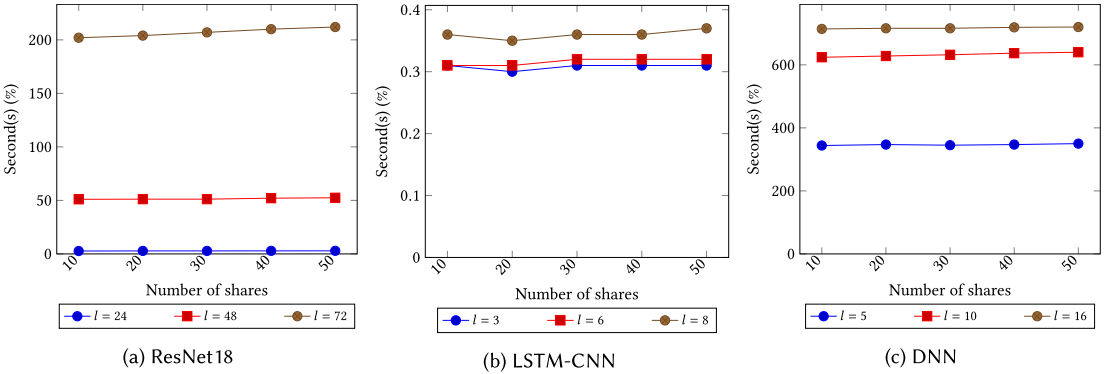


Fig. 10. Performance comparisons of the proposed method with state-of-the-art.

Fig. 11. Time taken for share generation based on different numbers of layers (l) in the NN model and the number of shares.

This is because as the number of layers increases, the number of values that clients need to generate shares for also increases. A similar trend is observed when the number of clients is increased. Although the increase in time is not substantial, there is a slight rise in execution time. This trend is most evident in Figure 11(a), where the number of layers is set to $l = 72$, and in Figure 11(c), where the number of layers is set to $l = 16$.

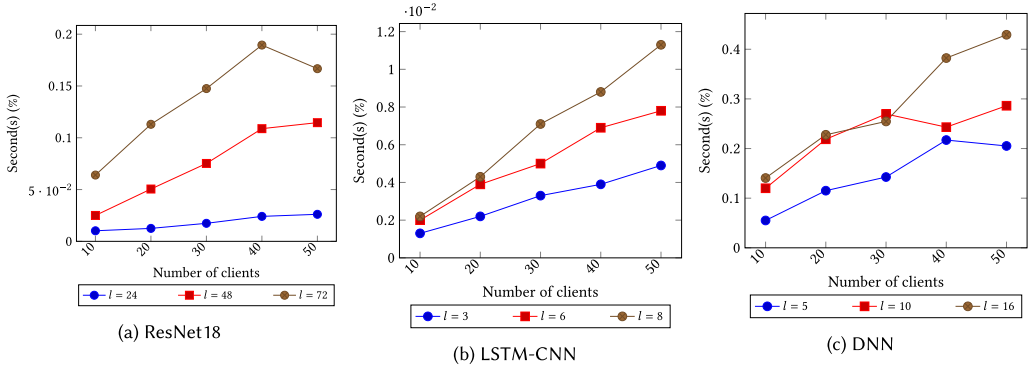


Fig. 12. Time taken for model aggregation based on different numbers of layers (l) in the NN model and the number of clients.

Table 2. Comparison of Communication Costs between the Proposed Method and Prior Works

Method	Comms. per client
FL [33]	$2 w_i + w_r $
SFL [38]	$2(w_i + \hat{O}_p n_p)$
SMPC-based FL [22]	$2K(w_i + w_r)$
APP-SplitFed	$2(K(w_i) + \hat{O}_p n_p)$

The aggregation time required exhibits a notable increase in execution time, as depicted in Figure 12. All machine learning models show a time increase with slight fluctuations during aggregation as the number of clients increases. Furthermore, increasing the number of layers to be aggregated prolongs the aggregation process across the three machine learning models tested in the experiment. Both increases are attributed to the greater number of values to be aggregated.

We evaluate the communication cost for each client using our proposed method and compare it with prior works, including FL [8], SFL [38], and SMPC-based aggregation in [22] for FL. The comparisons are presented in Table 2. To maintain consistency, we designate a learning model with two components: w_i and w_r . In the FL approach, the sum $w_i + w_r$ represents the full learning model.

As shown in the table, FL involves exchanging the full model between the client and the cloud server. This exchange includes the client sending the trained local model to the server and the cloud sending the aggregated local models back to the client. In SFL, we must consider the size of the intermediate output $\hat{O}_p$ exchanged between the client and server, multiplied by the number of data points n_p , and the size of the initial layer parameters w_i , doubled for the round trip. The work in [22] requires communication costs similar to FL, with additional costs for exchanging shares of the entire model instead of local model parameters. The number of shares depends on the number of clients K participating in the aggregation process. Therefore, the exchange of whole model parameters equals two times K times the size of $w_i + w_r$. In our approach, instead of generating shares for the entire model parameters, clients only generate shares for w_i , reducing communication costs by w_r times the number of shares K , times two. However, an additional exchange of the intermediate output $\hat{O}_p$ needs to be considered. If the size of the intermediate output $\hat{O}_p$ times the number of

data points n_p is less than the size of $2K|w_r|$, then the communication cost of APP-SplitFed is less than the proposed method in [22].

5 Future Work

Building upon the foundations laid by APP-SplitFed in SFL, there are several promising directions for further research and development. These advancements aim not only to enhance the capabilities of the current framework but also to address emerging challenges and opportunities in the rapidly evolving landscape of multimedia in smart healthcare systems and beyond. Here are some key areas for future work:

- *Scalability and Robustness*: Future studies should focus on scaling APP-SplitFed for larger and more complex multimedia-based smart healthcare services. It's crucial to assess how effectively the model performs under various network conditions and with an increased number of users. Specifically, there is a need to enhance the cloud server's capacity to handle the growing volume of data and requests. Additionally, adapting the model to process diverse multimedia healthcare data efficiently will be essential in ensuring robustness and scalability.
- *Enhanced Privacy Features*: While APP-SplitFed already includes privacy protection features, there is room for improvement. Future enhancements could involve implementing more advanced methods for securing and anonymizing data. Exploring newer encryption techniques and innovative anonymization methods will be critical in safeguarding sensitive multimedia healthcare data as it is processed and transmitted across the network.
- *System Flexibility*: The current version of APP-SplitFed assumes uniform computational tasks for all client devices. Future work should explore customizing learning tasks based on each device's specific capabilities to ensure effective contributions from all devices, regardless of their individual power and resources. This adaptation could involve dynamically assigning tasks tailored to the computational strengths of each device, thereby optimizing the overall system performance and efficiency in processing multimedia healthcare data.

6 Conclusion

We introduce APP-SplitFed, a secure, asynchronous partial weighted model aggregation method for SFL within multimedia healthcare service environments. APP-SplitFed ensures model privacy through SMPC, concealing vital information within a learning model's initial layers that is crucial for healthcare data reconstruction when uploaded to the cloud. The split model in APP-SplitFed enables resource-constrained multimedia health monitoring devices within smart healthcare systems to participate in decentralized machine learning. It also provides efficient resource usage by allowing clients with more capability to continue the process without being delayed by clients with fewer resources. Additionally, the proposed asynchronous model update in splitfed allows models to converge faster. The effectiveness of the proposed APP-SplitFed has also been compared to existing methods, and the results show that our proposed method outperformed the existing works.

References

- [1] Andrea Acevedo, Anna Merino, Santiago Alf  rez,   ngel Molina, Laura Bold  , and Jos   Rodellar. 2020. A dataset of microscopic peripheral blood cell images for development of automatic recognition systems. *Data in Brief* 30 (2020), 105474.
- [2] Oresti Banos, Rafael Garcia, Juan A. Holgado-Terriza, Miguel Damas, Hector Pomares, Ignacio Rojas, Alejandro Saez, and Claudia Villalonga. 2014. mHealthDroid: A novel framework for agile development of mobile health applications. In *Proceedings of the International Workshop on Ambient Assisted Living*. Springer, 91–98.
- [3] Oresti Banos, Claudia Villalonga, Rafael Garcia, Alejandro Saez, Miguel Damas, Juan A Holgado-Terriza, Sungyong Lee, Hector Pomares, and Ignacio Rojas. 2015. Design, implementation and validation of a novel open framework for agile development of mobile health applications. *Biomedical Engineering Online* 14, 2 (2015), 1–20.

- [4] Michele Bernardini, Luca Romeo, Emanuele Frontoni, and Massih-Reza Amini. 2021. A semi-supervised multi-task learning approach for predicting short-term kidney disease evolution. *IEEE Journal of Biomedical and Health Informatics* 25, 10 (2021), 3983–3994. DOI: <https://doi.org/10.1109/JBHI.2021.3074206>
- [5] Jordan J. Bird, Aniko Ekart, Christopher D. Buckingham, and Diego R. Faria. 2019. Mental emotional sentiment classification with an EEG-based brain-machine interface. In *Proceedings of the International Conference on Digital Image and Signal Processing (DISP '19)*.
- [6] Jordan J. Bird, Diego R. Faria, Luis J. Manso, Anikó Ekárt, and Christopher D. Buckingham. 2019. A deep evolutionary approach to bioinspired classifier optimisation for brain-machine interaction. *Complexity* 2019 (2019), 4316548.
- [7] Victoria Blanes-Vidal, Gunnar Baatrup, and Esmaeil S. Nadimi. 2018. Machine learning-based colorectal cancer detection. In *Proceedings of the Conference on Research in Adaptive and Convergent Systems (RACS '18)*. ACM, New York, NY, 43–46. DOI: <https://doi.org/10.1145/3264746.3264785>
- [8] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 1175–1191.
- [9] Ming Chen, Bingcheng Mao, and Tianyi Ma. 2021. FedSA: A staleness-aware asynchronous Federated Learning algorithm with non-IID data. *Future Generation Computer Systems* 120 (2021), 1–12.
- [10] Yang Chen, Xiaoyan Sun, and Yaochu Jin. 2020. Communication-efficient federated deep learning with layerwise asynchronous model update and temporally weighted aggregation. *IEEE Transactions on Neural Networks and Learning Systems* 31, 10 (2020), 4229–4238. DOI: <https://doi.org/10.1109/TNNLS.2019.2953131>
- [11] Zheyi Chen, Weixian Liao, Kun Hua, Chao Lu, and Wei Yu. 2021. Towards asynchronous federated learning for heterogeneous edge-powered Internet of Things. *Digital Communications and Networks* 7, 3 (2021), 317–326.
- [12] Ronald Cramer, Ivan Bjerre Damgård, and Jesper Buus Nielsen. 2015. *Secure Multiparty Computation*. Cambridge University Press.
- [13] Yongheng Deng, Feng Lyu, Ju Ren, Yi-Chao Chen, Peng Yang, Yuezhi Zhou, and Yaoxue Zhang. 2022. Improving federated learning with quality-aware user incentive and auto-weighted model aggregation. *IEEE Transactions on Parallel and Distributed Systems* 33, 12 (2022), 4515–4529.
- [14] Bin Gu, An Xu, Zhouyuan Huo, Cheng Deng, and Heng Huang. 2022. Privacy-preserving asynchronous vertical federated learning algorithms for multiparty collaborative learning. *IEEE Trans. Neural Netw. Learn. Syst.* 33, 11 (Nov 2022), 6103–6115. DOI: <https://doi.org/10.1109/TNNLS.2021.3072238> 34161243
- [15] Otkrist Gupta and Ramesh Raskar. 2018. Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications* 116 (2018), 1–8.
- [16] Jiangshan Hao, Yanchao Zhao, and Jiale Zhang. 2020. Time efficient federated learning with semi-asynchronous communication. In *Proceedings of the IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 156–163.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.
- [19] Chung-Hsuan Hu, Zheng Chen, and Erik G Larsson. 2023. Scheduling and aggregation design for asynchronous federated learning over wireless networks. *IEEE Journal on Selected Areas in Communications* 41, 4 (2023), 874–886.
- [20] Bin Jia, Xiaosong Zhang, Jiewen Liu, Yang Zhang, Ke Huang, and Yongquan Liang. 2022. Blockchain-enabled federated learning data protection aggregation scheme with differential privacy and homomorphic encryption in IIoT. *IEEE Transactions on Industrial Informatics* 18, 6 (2022), 4049–4058. DOI: <https://doi.org/10.1109/TII.2021.3085960>
- [21] Aditya Pribadi Kalapaaking, Ibrahim Khalil, Mohammad Saidur Rahman, Mohammed Atiquzzaman, Xun Yi, and Mahathir Almashor. 2023. Blockchain-based federated learning with secure aggregation in trusted execution environment for internet-of-things. *IEEE Trans. Ind. Inf.* 19, 2 (2023), 1703–1714. DOI: <https://doi.org/10.1109/TII.2022.3170348>
- [22] Aditya Pribadi Kalapaaking, Veronika Stephanie, Ibrahim Khalil, Mohammed Atiquzzaman, Xun Yi, and Mahathir Almashor. 2022. SMPC-based federated learning for 6G-enabled Internet of medical things. *IEEE Network* 36, 4 (2022), 182–189. DOI: <https://doi.org/10.1109/MNET.007.2100717>
- [23] Renuga Kanagavelu, Qingsong Wei, Zengxiang Li, Haibin Zhang, Juniarto Samsudin, Yechao Yang, Rick Siow Mong Goh, and Shuangguang Wang. 2022. CE-Fed: Communication efficient multi-party computation enabled federated learning. *Array* 15 (2022), 100207. DOI: <https://doi.org/10.1016/j.array.2022.100207>
- [24] Jakob Nikolas Kather, Johannes Krisam, Pornpimol Charoentong, Tom Luedde, Esther Herpel, Cleo-Aron Weis, Timo Gaiser, Alexander Marx, Nektarios A. Valous, Dyke Ferber, Lina Jansen, Constantino Carlos Reyes-Aldasoro, Inka Zörnig, Dirk Jäger, Hermann Brenner, Jenny Chang-Claude, Michael Hoffmeister, and Niels Halama. 2019. Predicting survival from colorectal cancer histology slides using deep learning: A retrospective multicenter study. *PLoS Medicine* 16, 1 (2019), e1002730.

- [25] Anastasiia Koloskova, Sebastian U Stich, and Martin Jaggi. 2022. Sharper convergence guarantees for asynchronous sgd for distributed and federated learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 17202–17215.
- [26] Hyun-Suk Lee and Jang-Won Lee. 2021. Adaptive transmission scheduling in wireless networks for asynchronous federated learning. *IEEE Journal on Selected Areas in Communications* 39, 12 (2021), 3673–3687. DOI: <https://doi.org/10.1109/JSAC.2021.3118353>
- [27] Seunghyun Lee, Jiwon Kim, Sung Woon Park, Sang-Man Jin, and Sung-Min Park. 2021. Toward a fully automated artificial pancreas system using a bioinspired reinforcement learning design: In silico validation. *IEEE Journal of Biomedical and Health Informatics* 25, 2 (2021), 536–546. DOI: <https://doi.org/10.1109/JBHI.2020.3002022>
- [28] Yinghui Liu, Youyang Qu, Chenhao Xu, Zhicheng Hao, and Bruce Gu. 2021. Blockchain-enabled asynchronous federated learning in edge computing. *Sensors* 21, 10 (2021), 3335.
- [29] Guodong Long, Ming Xie, Tao Shen, Tianyi Zhou, Xianzhi Wang, and Jing Jiang. 2023. Multi-center federated learning: Clients clustering for better personalization. *World Wide Web* 26, 1 (2023), 481–500.
- [30] Yunlong Lu, Xiaohong Huang, Ke Zhang, Sabita Maharjan, and Yan Zhang. 2021. Communication-efficient federated learning for digital twin edge networks in industrial IoT. *IEEE Transactions on Industrial Informatics* 17, 8 (2021), 5709–5718. DOI: <https://doi.org/10.1109/TII.2020.3010798>
- [31] Milad Nasr, Reza Shokri, and Amir Houmansadr. 2019. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 739–753. DOI: <https://doi.org/10.1109/SP.2019.00065>
- [32] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. 2022. Federated learning with buffered asynchronous aggregation. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*. PMLR, 3581–3607.
- [33] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguerre y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, Vol. 54, 1273–1282.
- [34] Lu Shi, Jiangang Shu, Weizhe Zhang, and Yang Liu. 2021. HFL-DP: Hierarchical federated learning with differential privacy. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, 1–7. DOI: <https://doi.org/10.1109/GLOBECOM46510.2021.9685644>
- [35] Ekanat Sotthiwat, Liangli Zhen, Zengxiang Li, and Chi Zhang. 2021. Partially encrypted multi-party computation for federated learning. In *Proceedings of the IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 828–835. DOI: <https://doi.org/10.1109/CCGrid51090.2021.00101>
- [36] Veronika Stephanie, Ibrahim Khalil, and Mohammed Atiquzzaman. 2023. Digital twin enabled asynchronous splitfed learning in e-healthcare systems. *IEEE Journal on Selected Areas in Communications* 41, 11 (2023), 3650–3661. DOI: <https://doi.org/10.1109/JSAC.2023.3310103>
- [37] Veronika Stephanie, Ibrahim Khalil, Mohammad Saidur Rahman, and Mohammed Atiquzzaman. 2023. Privacy-preserving ensemble infused enhanced deep neural network framework for edge cloud convergence. *IEEE Internet of Things Journal* 10, 5 (2023), 3763–3773.
- [38] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. 2022. Splitfed: When federated learning meets split learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36, 8485–8493.
- [39] Qizhao Wang, Qing Li, Kai Wang, Hong Wang, and Peng Zeng. 2021. Efficient federated learning for fault diagnosis in industrial cloud-edge computing. *Computing* 103, 10 (2021), 2319–2337.
- [40] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H. Yang, Farhad Farokhi, Shi Jin, Tony Q. S. Quek, and H. Vincent Poor. 2020. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3454–3469. DOI: <https://doi.org/10.1109/TIFS.2020.2988575>
- [41] Wentai Wu, Ligang He, Weiwei Lin, Rui Mao, Carsten Maple, and Stephen Jarvis. 2020. SAFA: A semi-asynchronous protocol for fast federated learning with low overhead. *IEEE Transactions on Computers* 70, 5 (2020), 655–668.
- [42] Chenhao Xu, Youyang Qu, Yong Xiang, and Longxiang Gao. 2023. Asynchronous federated learning on heterogeneous devices: A survey. *Computer Science Review* 50 (2023), 100595.
- [43] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. 2023. MedMNIST v2-a large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data* 10, 1 (2023), 41.
- [44] Li Zhang, Jianbo Xu, Pandi Vijayakumar, Pradip Kumar Sharma, and Uttam Ghosh. 2022. Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system. *IEEE Transactions on Network Science and Engineering* (2022), 1–17. DOI: <https://doi.org/10.1109/TNSE.2022.3185327>
- [45] Yu Zhang, Morning Duan, Duo Liu, Li Li, Ao Ren, Xianzhang Chen, Yajuan Tan, and Chengliang Wang. 2021. CSAFL: A clustered semi-asynchronous federated learning framework. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–10.

- [46] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2020. iDLG: Improved deep leakage from gradients. arXiv:2001.02610. DOI: <https://doi.org/10.48550/arXiv.2001.02610>
- [47] Chendi Zhou, Hao Tian, Hong Zhang, Jin Zhang, Mianxiong Dong, and Juncheng Jia. 2021. TEA-fed: Time-efficient asynchronous federated learning for edge computing. In *Proceedings of the 18th ACM International Conference on Computing Frontiers*, 30–37.
- [48] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*.

Received 14 March 2024; revised 19 June 2024; accepted 3 September 2024